# GitHub Copilot Adoption Framework

導入計画・ガバナンス・定着化を体系化する実践ガイド

2026-03-21

# GitHub Copilot Adoption Framework

**導入・運用・ガバナンス実践ガイド**

企業・組織全体で GitHub Copilot を安全かつ効果的に導入・運用するための、包括的なフレームワーク教材です。

> [Note] ブラウザで https://duwenji.github.io/spa-quiz-app/ を開くと、関連トピックをクイズ形式で復習できます。

- 著者: 杜 文吉
- 対象: CTO / 推進担当 / Tech Lead / セキュリティ担当
- テーマ: adoption / governance / KPI / operating model

**この教材で学べること** - 組織導入ロードマップと KPI 設計 - ガバナンスと利用ルールの整備 - 現場展開と Platform / Security 観点の整理 - 継続運用のための標準化と改善サイクル

# 目次


- GCAF（GitHub Copilot Adoption Framework）v1.0 (#chapter-01-framework)
  - GCAF（GitHub Copilot Adoption Framework）v1.0 (#section-01-framework-01-overview)
- Governance (#chapter-02-governance)
  - Copilot利用ポリシーひな型（禁止事項・責任分界・例外手続き） (#section-02-governance-01-policy-template)
  - Content Exclusion 推奨パターン集（GitHub Copilot向け） (#section-02-governance-02-content-exclusion)
- Operations (#chapter-03-operations)
  - レビュー観点チェックリスト（GitHub Copilot利用時） (#section-03-operations-01-review-checklist)
  - チャット参加者・スラッシュコマンド・チャット変数の詳細リスト (#section-03-operations-02-chat-reference)

# GCAF（GitHub Copilot Adoption Framework）v1.0

## GCAF（GitHub Copilot Adoption Framework）v1.0

> **関連ドキュメント**: ポリシーひな型 (#section-02-governance-01-policy-template) | Content Exclusion推奨パターン集 (#section-02-governance-02-content-exclusion) | レビュー観点チェックリスト (#section-03-operations-01-review-checklist) | チャット機能詳細リスト (#section-03-operations-02-chat-reference)

### 目次


- 0. エグゼクティブサマリ (#0-エグゼクティブサマリ)
- 1. CAF対応：GCAFの6つの視点（Perspectives） (#1-caf対応gcafの6つの視点perspectives)
- 2. KPI / メトリクス（測定設計） (#2-kpi--メトリクス測定設計)
- 3. Copilot Metrics（詳細） (#3-copilot-metrics詳細)
- 4. 導入フェーズ（Phases）：GCAFの標準ロードマップ (#4-導入フェーズphasesgcafの標準ロードマップ)
- 5. ガードレール（GCAF標準ポリシーひな型） (#5-ガードレールgcaf標準ポリシーひな型)
- 6. Enablement（教育・定着の標準パッケージ） (#6-enablement教育定着の標準パッケージ)
- 7. 最小実装チェックリスト（MRT：Minimum Required Tasks） (#7-最小実装チェックリストmrtminimum-required-tasks)
- 8. 用語リスト（Glossary） (#8-用語リストglossary)
- 付録A：参照資料（公式・準公式・ベンダー提供） (#付録a参照資料公式準公式ベンダー提供)
- 次にできること（おすすめ） (#次にできることおすすめ)
- 質問（最小限） (#質問最小限)
- Image一覧 (#image一覧)

## 0. エグゼクティブサマリ

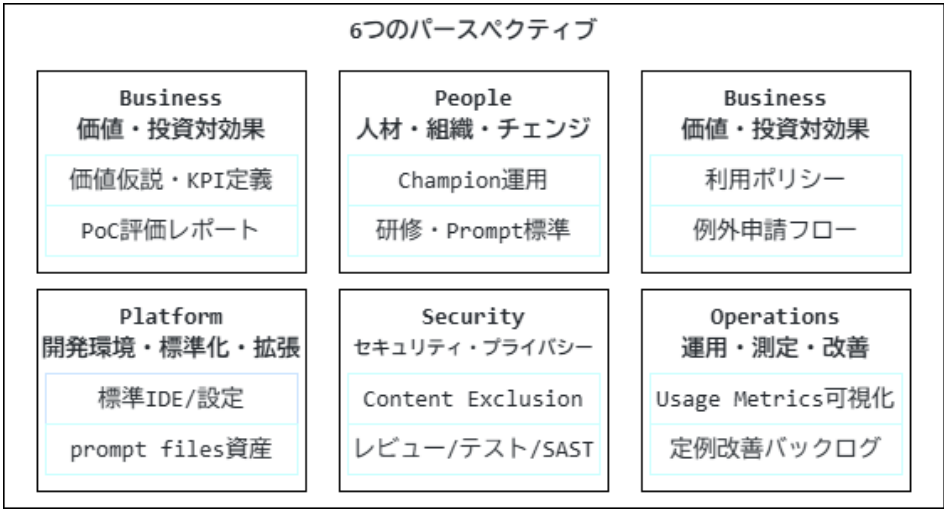
- **目的**：Copilotを"開発者の拡張（Augmentation）"として定着させ、**品質・速度・開発者体験**を同時に改善する。GitHub Copilotは、開発者の生産性を向上させるためのAIツールであり、コードの提案や補完を通じて開発プロセスを効率化します。これにより、開発者は反復的な作業から解放され、より創造的なタスクに集中できるようになります。さらに、導入後の効果を測定するための指標やベストプラクティスも提供されており、組織全体での価値を最大化することが可能です。measuring-the-impact-of-github-copilot (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)
- **導入原則**：
  1. **段階導入**（Pilot -> Scale）で学習しながら広げる。段階的な導入は、初期段階での学びを活かし、リスクを最小限に抑えながらスムーズに拡大するための重要な戦略です。これにより、組織はCopilotの効果を実証し、必要に応じてプロセスを調整することができます。enable-developers/drive-adoption (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), copilot-adoption (https://samqbush.github.io/copilot-adoption/)
  2. **ガバナンス先行**（ポリシー/除外/監査/責任分界）。ガバナンスを先行させることで、Copilotの利用に伴うリスクを事前に管理し、組織全体での信頼性を確保します。これには、利用ポリシーの策定やコンテンツ除外ルールの設定が含まれます。manage-enterprise-policies (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies), content-exclusion (https://docs.github.com/en/copilot/concepts/context/content-exclusion), copilot-trust-center (https://resources.github.com/ja/copilot-trust-center/)
  3. **メトリクスで改善**（使用状況・受入率・PRサイクル）。メトリクスを活用することで、Copilotの導入効果を定量的に評価し、継続的な改善を図ることができます。これにより、開発プロセスの効率化や品質向上が実現します。copilot-metrics (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics), view-usage-and-adoption (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption), measuring-the-impact-of-github-copilot (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)

- **成果物**：ポリシー、利用ガイド、教育、環境標準、KPIダッシュボード、改善バックログ。これらの成果物は、Copilotの導入と運用を成功させるための基盤となります。具体的には、利用ポリシーの策定や教育プログラムの提供、KPIダッシュボードを活用した効果測定が含まれます。roll-out-at-scale (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), copilot-usage-metrics (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics), prompt-engineering (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering)

---

## 1. CAF対応：GCAFの6つの視点（Perspectives）

AWS CAFの「観点」をCopilotに置き換え、**6視点×成果物**で標準化します。GitHub側にも“導入・測定・設計原則”が分散資料として存在するため、それを統合してCAF相当の枠組みにします。日常開発へ落とし込む具体的な標準フローは、補完教材の 開発プロセス標準化 Skill ライブラリ (https://github.com/duwenji/dev-process-skill-library) と組み合わせる想定です。wellarchitected.github.com (https://wellarchitected.github.com/features/copilot/), enable-developers/drive-adoption (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), measuring-the-impact-of-github-copilot (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)




6視点の1枚絵（CAFライク）

### 1) Business（価値・投資対効果）

**ゴール**：経営/管理層が投資判断できるように、価値仮説と測定方法を定義する。

**主要論点** GitHub Copilotの導入効果を測定するための指標として、開発速度の向上（55%のタスク解決時間短縮）、コードレビューの効率化（67%の高速化）、コード品質の向上（85%の信頼性向上）が挙げられます。これらの指標は、開発者の生産性向上やチームのモチベーション維持に寄与します。また、GitHub Copilot Chatを活用することで、リアルタイムでのコードレビューやフィードバックが可能となり、開発プロセス全体の効率化が期待されます。さらに、GitHub Copilotの使用状況を可視化するダッシュボードやAPIを活用することで、導入効果を定量的に把握し、経営層への説明資料として活用できます。

**成果物（Artifacts）**

- 価値仮説・KPI定義書（後述のKPIセット）
- PoC評価レポート（Before/After）
- 導入判断メモ（リスク・費用・効果）

---

**2) People（人材・組織・チェンジマネジメント）**

**ゴール：**AIと協働できる開発者を増やし、学習曲線と抵抗感を下げる。

**主要論点** GitHub Copilotの導入を成功させるためには、チャンピオンプログラムの運用やワークショップの開催が重要です。具体的には、導入前に成功指標を定義し、チャンピオンを育成することで、社内での普及を促進します。また、プロンプトエンジニアリングの基本を学ぶことで、開発者がCopilotを効果的に活用できるようになります。さらに、オンボーディング資料やFAQ、トラブルシューティングガイドを整備することで、導入初期の課題を軽減します。これにより、開発者がAIツールを活用するスキルを向上させ、組織全体の生産性を高めることができます。

**成果物**

- Copilot 利用ガイド（開発者版）
- Champion Program運用（FAQ、相談窓口、Tips配信）
- 研修（基礎：Prompt/レビュー/セキュア、応用：テスト生成/リファクタ/設計相談）
- “Promptパターン集”（例：テスト生成、既存コード説明、リファクタ方針、脆弱性観点レビュー）

---

**3) Governance（ポリシー・知財・コンプライアンス）**

**ゴール：**「使っていい/悪い」を明確にして、リスクを事前に潰す。

**主要論点** GitHub Copilotの利用におけるリスク管理には、エンタープライズポリシーの策定が不可欠です。具体的には、機密情報や規制対象データをContent Exclusion機能で除外することで、AIが不適切なデータを参照するリスクを軽減します。また、Trust Centerを活用して、セキュリティやプライバシーに関する透明性を確保し、経営層や法務部門への説明責任を果たします。さらに、AI生成コードのレビュー基準を明確化し、通常のコードと同等の品質管理を行うことで、開発プロセス全体の信頼性を向上させます。

**成果物**

- Copilot利用ポリシー（目的、対象範囲、禁止事項、責任分界点）
- コンテンツ除外ルール（パス/拡張子/リポジトリ単位の方針）
- レビュー標準（AI生成コードも"通常のコード"として同等レビュー）
- 例外申請フロー（高リスク領域での利用手続き）

---

### 4) Platform（開発環境・標準化・拡張）

**ゴール**：Copilotが効果を出しやすい環境とリポジトリ標準を整える。

**主要論点** GitHub Copilotの効果を最大化するためには、開発環境の標準化と拡張が重要です。具体的には、セットアップやトラブルシューティングの手順を明確化し、運用の効率化を図ります。また、Copilot Chatの活用方法を標準化することで、チーム全体の生産性を向上させます。さらに、VS Codeのprompt files（.prompt.md）を活用して、よく使う作業をテンプレート化し、社内でのプロンプト資産を構築します。これにより、開発者が効率的に作業を進められる環境を提供します。

**成果物**

- 標準IDE/拡張構成（推奨設定、社内テンプレ）
- リポジトリ標準（README、CONTRIBUTING、テスト方針、Lint/Format）
- プロンプト資産（prompt files、ガイド、例題集）

---

### 5) Security（セキュリティ・プライバシー・サプライチェーン）

**ゴール**：データ取り扱いと生成コードの安全性を担保する。

**主要論点** GitHub Copilotのセキュリティを確保するためには、Trust Centerを活用して暗号化やデータ保持に関する透明性を確保することが重要です。また、Content Exclusion機能を利用して機密情報がAIの学習対象にならないように設定することで、データ漏洩リスクを軽減します。さらに、生成コードのレビューやテスト、SAST（静的解析ツール）を活用して、セキュリティリスクを早期に発見・対応する体制を整えます。これにより、開発プロセス全体の安全性を向上させることができます。

**成果物**

- セキュリティ利用ガイド（入力禁止：秘密情報、個人情報、鍵、顧客データ等）
- "AI生成コード"レビュー観点チェックリスト（認可/暗号/入力検証/依存関係/例外処理）
- 監査・証跡（ポリシー設定の記録、運用ログ方針）

---

### 6) Operations（運用・測定・改善）

**ゴール**：導入して終わりではなく、データで定着と改善を回す。

**主要論点** GitHub Copilotの運用を最適化するためには、Copilot usage metricsを活用して採用状況や利用状況を可視化することが重要です。これにより、導入効果を定量的に把握し、改善点を特定できます。また、定期的なChampion会議や品質会議を通じて、現場からのフィードバックを収集し、改善バックログに反映させる仕組みを構築します。さらに、PoC（概念実証）から全社展開までの振り返りを行い、成功事例や課題を共有することで、継続的な改善サイクルを実現します。

**成果物**

- Copilot運用ダッシュボード（公式メトリクス＋社内BI）
- 定例（Champion会/品質会/改善バックログ）
- PoC -> 全社展開の振り返りテンプレ

---

## 2. KPI / メトリクス（測定設計）

「生産性は測りにくい」前提で、**テレメトリ**＋**サーベイ**＋**成果指標**を組み合わせます（公式もこの立場）。resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/), docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics)

### 2.1 Leading Indicators（先行指標：早期に効く）

- アクティブユーザー、利用頻度、受入率、言語/IDE別傾向（Copilot usage metrics） docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics), github.blog (https://github.blog/changelog/2025-10-28-copilot-usage-metrics-dashboard-and-api-in-public-preview/)
- 開発者アンケ（有用性/ブロッカー/満足度） resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)

### 2.2 Lagging Indicators（遅行指標：成果）

- PR throughput、time to merge 等（影響を見る） resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/), docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics)
- 品質（欠陥混入率、レビュー指摘、セキュリティ検知など） resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/), c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/)

### 2.3 計測基盤

- 公式：ダッシュボード/API/NDJSON（プレビューである点・IDEテレメトリ依存を明示） docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics), docs.github.com (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption), github.blog (https://github.blog/changelog/2025-10-28-copilot-usage-metrics-dashboard-and-api-in-public-preview/)
- 参考：ソリューションアクセラレータ（メトリクス可視化） github.com (https://github.com/microsoft/copilot-metrics-dashboard)

---

## 3. Copilot Metrics（詳細）

GitHub Copilot usage metricsは、Copilotの導入効果を可視化し、採用状況や利用パターンを追跡するデータセットです。以下の詳細情報を参考に、組織の導入効果を測定してください。

**Supported IDEs**

Copilot usage metricsはIDEからのテレメトリに基づくため、以下のサポート対象IDEを利用する必要があります： - **Visual Studio Code**: Version 1.101 以上 + Copilot Chat extension 0.28.0 以上 - **JetBrains IntelliJ**: Version 2024.2.6 以上 + Copilot extension 1.5.52-241 以上 - **Visual Studio**: Version 17.14.13 以上 + Copilot extension 18.0.471.29466 以上 - **Eclipse**: Version 4.31 以上 + Copilot extension 0.9.3.202507240902 以上 - **Xcode**: Version 13.2.1 以上 + Copilot extension 0.40.0 以上

**注意**: テレメトリが有効でないユーザーはメトリクスに計上されません。

**Data Freshness（データの鮮度）**

Copilot usage metricsダッシュボードおよびAPIは定期的に更新されます： - **標準**: データは発生から3暦日以内に利用可能（例：月曜日のデータは木曜日までに表示） - **遅延**: 週末を跨ぐ場合など、最大4暦日の遅延が生じる可能性があります - **プレビュー期間中**: 仕様変更の可能性があるため、本番依存には注意が必要です

**What Does the Data Measure?（測定内容）**

Copilot usage metricsは、以下の5つの主要カテゴリで構成されます：

**1) Adoption（採用率）** - Daily Active Users（DAU）: 特定の日にCopilotと相互作用したユニークユーザー数 - Weekly Active Users（WAU）: 週単位のユーザー数 - 初期段階では、この指標が継続的に上昇することが重要です

**2) Engagement（利用頻度・深さ）** - Average requests per active user: 平均的なユーザーがChat/Completionを利用した頻度 - Breadth across features: 複数の機能（Chat、Completions、Agents など）を利用しているか - 単なる利用者数ではなく、実際の使用パターンを反映した指標

**3) Acceptance Rate（受入率）** - Inline suggestions acceptance rate: エディタ提案の受け入れ率 - Chat follow-up rate: Chat回答への追加操作率 - **信頼性の指標**: 提案が実務的に有用かどうかを示す

**4) Lines of Code (LoC)（コード生成量）** - Lines suggested: Copilotが提案したコード行数 - Lines added: 実際に受け入れられ挿入されたコード行数 - Lines deleted: Copilot支援で削除されたコード行数 - **方向指標**: 生産性への定性的な見方を提供（定量的因果関係は限定的）

**5) Pull Request Lifecycle Metrics（PR関連指標）** - PR creation count: PR created by Copilot（準備中等） - PR merge count: マージされたPR数 - Median time to merge: 中央値マージ時間 - Code review suggestion activity: レビュー提案の活動量 - **成果指標**: Copilot

活用がデリバリースピード・品質に与える影響を測定

**How Can I Use These Metrics?（活用方法）**

これらのメトリクスを組み合わせて、以下の問いに答えることができます：

| 質問 | 参考メトリクス |
|---|---|
| チームはCopilotをコンスタントに利用しているか？ | Daily/Weekly Active Users |
| どの機能が最大の価値を提供しているか？ | Requests per feature, Agent adoption |
| 開発者はCopilotの出力を信頼しているか？ | Acceptance rate trends over time |
| トレーニングや施策は効果を上げているか？ | Adoption and engagement growth |
| Copilotはデリバリースピードに影響しているか？ | PR merge counts, median time to merge, cycle time |

**パターン分析のコツ**: - 単一の数字に注目するのではなく、複数の信号を組み合わせて解釈する - 例：DAUが安定 + 受入率が上昇 = 信頼度向上 + 実務的価値の確立 - トレンド（方向）を重要視し、絶対値よりも変化率に着目する

**計測基盤と制限**

**公式ツール**: - **ダッシュボード**: GitHub Enterprise/Organization管理画面で4週間（28日）の傾向を可視化 - **REST API**: プログラマティックアクセスで詳細なイベント テレメトリを取得 - **NDJSON Export**: 生データをBI/分析ツール用に抽出

**重要な制限**: - **IDE テレメトリ依存**: 他のCopilot surface（Web上のChat、Mobile、CLIなど）は含まれない - **ライセンス/シート管理データは別**: 計測されるのは「利用活動」のみで、「割り当て」ではない - **プレビュー状態**: 仕様や保持期間が変更される可能性あり

**参考資料**

詳細については、以下を参照してください： - GitHub Copilot usage metrics (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics) - Viewing the Copilot usage metrics dashboard (https://docs.github.com/en/copilot/how-tos/administer-

copilot/manage-for-enterprise/view-usage-and-adoption) - Measuring the impact of GitHub Copilot (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)

---

## 4. 導入フェーズ（Phases）：GCAFの標準ロードマップ

公式の"adoptionを駆動するプロセス"は段階導入を推奨しているため、GCAFもフェーズ制にします。docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), samqbush.github.io (https://samqbush.github.io/copilot-adoption/)

### Phase 0：Prepare（準備）

**目的**：導入目的・対象・統制・測定の土台を固める。*主タスク** [resources.github.com]

**目的**：導入目的・対象・統制・測定の土台を固める。

**主タスク** resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/), docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies), docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics)

- ユースケース選定（テスト生成、定型コード、リファクタなど）
- ポリシー草案（禁止事項、責任分界、除外、監査） docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies), docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion), copilot.gi...trust.page (https://copilot.github.trust.page/faq)
- KPIとベースライン（PR時間、開発者アンケ、使用率） resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/), docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics)

### Exit criteria（次へ進む条件）

- ポリシー承認（暫定で可）
- PoC設計（対象チーム、期間、評価方法）

---

### Phase 1：Pilot（小規模実証）

**目的**：安全に試し、学びを“標準”に落とす。

**主タスク** docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), samqbush.github.io (https://samqbush.github.io/copilot-adoption/)

- 小規模チームへ付与、環境セットアップ支援 docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption)
- トレーニング（プロンプト・レビュー・セキュア） docs.github.com (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering), learn.microsoft.com (https://learn.microsoft.com/en-us/training/modules/introduction-prompt-engineering-with-github-copilot/), c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/)
- コンテンツ除外（.env/鍵/契約資料など） docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion), devblogs.m...rosoft.com (https://devblogs.microsoft.com/cppblog/configure-github-copilot-access-via-content-exclusion/)
- メトリクス収集開始（usage metrics、アンケ） copilot-usage-metrics (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics), resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)

#### Exit criteria

- “やってよい/悪い”が現場で理解される
- KPI初期値と改善仮説が立つ

---

### Phase 2：Early Scale（拡大：複数チームへ）

**目的**：Championと標準テンプレで“自走”を作る。

**主タスク** docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), devblogs.m...rosoft.com (https://devblogs.microsoft.com/all-things-azure/adopting-github-copilot-at-scale/)

- Champion育成・コミュニティ化 docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), devblogs.m…rosoft.com (https://devblogs.microsoft.com/all-things-azure/adopting-github-copilot-at-scale/)
- 標準プロンプト資産（prompt engineering指針、prompt files） docs.github.com (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering), code.visua…studio.com (https://code.visualstudio.com/docs/copilot/customization/prompt-files)
- ルールの明文化（例外/高リスク領域の扱い） c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/), copilot.gi…trust.page (https://copilot.github.trust.page/faq)
- ダッシュボードでチーム差を特定し、支援を厚くする docs.github.com (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption), github.blog (https://github.blog/changelog/2025-10-28-copilot-usage-metrics-dashboard-and-api-in-public-preview/)

---

**Phase 3：Enterprise Scale（全社展開）**

**目的**：統制と可視化を“仕組み化”し、横展開する。

**主タスク** docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies), copilot-usage-metrics (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics), wellarchit…github.com (https://wellarchitected.github.com/features/copilot/)

- Enterprise AI Controls（ポリシー）を統一 docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies)
- Content Exclusionの標準テンプレ適用 docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion)
- 運用（ライセンス付与/ヘルプ/FAQ/教育）を定常化 docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption)
- メトリクス定例（採用率・受入率・PRサイクル） copilot-usage-metrics (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics), github.blog (https://github.blog/changelog/2025-10-28-copilot-usage-metrics-dashboard-and-api-in-public-preview/)

---

### Phase 4：Optimize（最適化・定着）

**目的**：測定 -> 改善 -> 再教育のループで効果を伸ばし続ける。

**主タスク** resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/), docs.github.com (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption)

- ESSPの考え方で障壁を特定し改善（品質/速度/幸福） resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)
- 低採用チームへの集中支援（プロンプト/環境/タスク設計）
- 成功パターンをプロンプト資産・テンプレに反映 code.visua...studio.com (https://code.visualstudio.com/docs/copilot/customization/prompt-files), github.com (https://github.com/github/awesome-copilot)

---

## 5. ガードレール（GCAF標準ポリシーひな型）

ここは社内の法務/セキュリティと合意しやすいよう、一次情報（Trust Center/公式Docs）を根拠にして構造化します。copilot.gi...trust.page (https://copilot.github.trust.page/faq), docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies), docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion)

### 5.1 利用範囲（推奨/条件付き/非推奨）

- **推奨**：テスト生成、ボイラープレート、リファクタ補助、ドキュメント下書き docs.github.com (https://docs.github.com/en/copilot/how-tos/chat-with-copilot/get-started-with-chat), resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)
- **条件付き**：認可/暗号/決済/安全性クリティカル（必ず人間の設計・レビュー・テスト） c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/), resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)

- **非推奨（原則禁止）**：秘密情報・個人情報・顧客データを含む入力（必要なら除外/ダミー化） docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion), resources.github.com (https://resources.github.com/ja/copilot-trust-center/)

### 5.2 コンテンツ除外（必須設定例）

- 例：`**/.env`、鍵/証明書、契約・要件文書、顧客データスキーマ、認可設定など
- Content Exclusionは、除外ファイルでの提案無効＋他ファイルへの影響遮断＋Chat/コードレビューにも影響する。docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion), devblogs.m...rosoft.com (https://devblogs.microsoft.com/cppblog/configure-github-copilot-access-via-content-exclusion/)

### 5.3 責任分界（必須明記）

- 生成物の最終責任は開発チーム（Copilotは補助）
- AI生成コードも通常のコードと同等のレビュー・テストを必須とする c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/), resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)

---

## 6. Enablement（教育・定着の標準パッケージ）

公式の“Drive adoption”の流れをベースに、社内運用に落とし込みます。docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), devblogs.m...rosoft.com (https://devblogs.microsoft.com/all-things-azure/adopting-github-copilot-at-scale/)

### 6.1 30/60/90日プラン（例）

- **30日**：Champion任命、基礎研修、FAQ/Slack/Teams窓口 docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), learn.microsoft.com (https://learn.microsoft.com/en-us/training/modules/introduction-prompt-engineering-with-github-copilot/)
- **60日**：ユースケース別ワークショップ（テスト/リファクタ/設計相談） docs.github.com (https://docs.github.com/en/copilot/how-tos/chat-with-copilot/get-started-with-chat), docs.github.com (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering)

- **90日**：メトリクスレビュー会、成功事例共有、プロンプト資産化 docs.github.com (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption), code.visua...studio.com (https://code.visualstudio.com/docs/copilot/customization/prompt-files)

### 6.2 プロンプト標準（GCAF Prompting Standard）

- 「最初に目的 -> 制約 -> 例 -> 分割 -> 曖昧さ排除」など、GitHub Docsの推奨に沿う docs.github.com (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering), github.blog (https://github.blog/developer-skills/github/how-to-write-better-prompts-for-github-copilot/)
- Chat利用の基本（@workspace、#fileなど）を標準化 docs.github.com (https://docs.github.com/en/copilot/how-tos/chat-with-copilot/get-started-with-chat), docs.github.com (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering)

---

## 7. 最小実装チェックリスト（MRT：Minimum Required Tasks）

「最初から作り込みすぎない」ために、チェックリストで最低限を定義します。下記は“Minimum Required Tasks for Copilot Adoption”の思想を取り込み、GCAFとして再構成しています。samqbush.github.io (https://samqbush.github.io/copilot-adoption/), docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption)

### 7.1 Pilot前に必須（Must）

- ポリシー（AI Controls）方針がある docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies)
- Content Exclusion（機密ファイル）設計 docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion), devblogs.m...rosoft.com (https://devblogs.microsoft.com/cppblog/configure-github-copilot-access-via-content-exclusion/)
- KPI/ベースライン定義（測定可能な範囲を明確化） resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/), docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics)
- 教育（プロンプト・レビュー・セキュリティ） learn.microsoft.com (https://learn.microsoft.com/en-us/training/modules/introduction-prompt-engineering-with-github-copilot/), docs.github.com

(https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering), c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/)

### 7.2 Scale前に必須（Must）

- Champion運用（支援導線） docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), devblogs.m...rosoft.com (https://devblogs.microsoft.com/all-things-azure/adopting-github-copilot-at-scale/)
- メトリクスダッシュボードで採用状況を確認 docs.github.com (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption), docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics)
- 例外申請・高リスク領域のガードレールが運用されている c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/), copilot.gi...trust.page (https://copilot.github.trust.page/faq)

---

## 8. 用語リスト（Glossary）

- **GCAF（GitHub Copilot Adoption Framework）**：本資料で定義した、Copilot導入のための実務フレームワーク。
- **CAF（Cloud Adoption Framework）**：導入を複数視点で整理する枠組み。本資料では考え方をCopilot導入に適用。
- **PoC（Proof of Concept）**：本格導入前に有効性やリスクを小規模で検証する実証。
- **KPI（Key Performance Indicator）**：導入効果を測るための重要指標。
- **ESSP**：GitHub Resourcesで示される、Copilot効果測定の考え方（段階的に評価・最適化するアプローチ）。
- **Champion**：現場展開をリードする推進担当者（相談窓口・事例共有・教育支援）。
- **AI Controls**：GitHub Enterprise/Organizationで設定するCopilotのポリシー制御。
- **Content Exclusion**：Copilotの提案・Chat文脈から特定コンテンツを除外する機能。
- **Prompt Engineering**：目的・制約・文脈を明確にして、回答品質を高めるプロンプト設計手法。
- **prompt files（.prompt.md）**：VS Code等で再利用可能なプロンプトテンプレート資産。

- **@workspace / #file**：Copilot Chatでワークスペース全体や特定ファイルを文脈指定するための記法。
- **SAST（Static Application Security Testing）**：静的解析でコードの脆弱性を検出する手法。
- **Leading Indicators（先行指標）**：導入初期に変化を捉えやすい指標（利用率・受入率など）。
- **Lagging Indicators（遅行指標）**：結果として現れる指標（PRサイクル・品質指標など）。
- **NDJSON（Newline Delimited JSON）**：1行1JSONで構成されるデータ形式。メトリクス出力の取り回しに利用。
- **MRT（Minimum Required Tasks）**：導入時に最低限実施すべき必須タスク群。
- **Copilot EMU (Enterprise Managed Users)**: 企業が自社のIDプロバイダー（Okta, Microsoft Entra ID等）を通じて作成・管理する専用アカウントで、GitHub Copilotを利用する運用形態。GitHub Enterprise Cloudの法人向け高度管理機能（EMU）の一部。
    - 主な特徴:
        - 一元的なアイデンティティ管理: 社内アカウントでシングルサインオン（SSO）が可能。退職・異動時のアクセス権削除もIdP側で即座に反映される。
        - セキュリティの強化: アカウントが企業テナント内に完全に隔離されるため、リポジトリへのアクセス制限やセキュリティポリシーを強力に適用可能。
        - コンプライアンス管理: 管理者が全ユーザーのCopilot利用状況やポリシー設定（コードスニペットの再利用許可など）を一括で統制できる。
        - 外部との隔離: 原則として個人のGitHubアカウントとは切り離された「企業専用アカウント」として機能し、機密情報の保護を優先した設計。
    - 導入メリット: 大規模組織におけるライセンス管理の自動化、セキュリティ・ガバナンスの向上、および監査対応の容易化。
- **GitHub Copilot code review**: GitHub Copilotが提供する、プルリクエスト（PR）やエディタ上のコードに対して、バグの検出、パフォーマンス改善、セキュリティの脆弱性チェックを自律的に行うAIエージェント。
    - 主な特徴:

        - 自律的なレビュー: PR作成時に自動で起動し、変更差分を分析して人間のようなコメント（レビュー）を残す。
        - リポジトリ全体の理解: 個別のファイルだけでなく、プロジェクト全体のコンテキストに基づいた高度なフィードバックを行う。
        - カスタム指示: .github/copilot-instructions.md 等のファイルを通じて、プロジェクト固有のコーディング規約や設計ルールを学習させることが可能。
        - マルチプラットフォーム: GitHub.com 上でのPRレビューに加え、VS Code などのIDE上でもコミット前のセルフチェックとして利用できる。
    - 導入メリット: レビューサイクルの高速化、人的ミスの削減、およびチーム内のコード品質の標準化。
- **seat management data（シート管理データ）**: GitHub Copilotのライセンス（シート）の割り当て状況や利用状況に関するデータ。管理者がGitHubの管理画面やAPIを通じて確認できる。
    - 主なデータ内容:
        - 割り当て済みのユーザー: 現在誰がシートを保有しているかのリスト。
        - 最終アクティビティ日 (last_activity_at): ユーザーが最後にCopilotを（IDE等で）利用した日時。
        - 利用状況メトリクス: 提案の受け入れ率やチャットの使用回数など、投資対効果（ROI）を測定するための統計データ。
        - 非アクティブなシート: 割り当てられているが一定期間利用がないアカウントの特定。
    - 活用シーン:
        - コスト最適化: 利用していないユーザーのシートを回収し、必要なメンバーに再割り当てする。
        - ガバナンス: 組織全体の利用傾向を把握し、導入の効果測定やセキュリティポリシーの適用状況を確認する。
- **SCIM（System for Cross-domain Identity Management）**: 複数のクラウドサービスやドメイン間で、ユーザーの作成・更新・削除（プロビジョニング）を自動化するためのオープン標準プロトコル。
    - GitHub Copilotにおける役割:

    - 自動プロビジョニング: 社内ディレクトリ（IdP）に新入社員を追加すると、GitHubアカウントとCopilotライセンスが自動的に作成・付与される。
    - 自動停止（デプロビジョニング）: 従業員が退職して社内アカウントが無効になると、GitHubのアクセス権やCopilotのシートも即座に自動回収される。
    - 属性同期: 氏名やメールアドレス、所属グループなどの変更がリアルタイムでGitHub側に同期される。
  - 主なメリット:
    - 運用の効率化: 管理者がGitHubの管理画面で一人ずつユーザーを追加・削除する手間がなくなる。
    - セキュリティ向上: 退職者のアカウント削除漏れを防ぎ、不要なアクセス権が残るリスクを排除できる。
    - ライセンスコストの最適化: 利用資格がなくなったユーザーから自動でシートを回収できる。
- **Git Credential Manager (GCM)**: Git操作（clone, push, pullなど）における認証プロセスを簡略化・セキュアにするためのオープンソースの資格情報ヘルパー。
  - 主な特徴:
    - 多要素認証（MFA）のサポート: ブラウザベースのログインを介して、GitHubの2要素認証やSSO（SAML/EMU環境）をスムーズに処理できる。
    - 安全な保存: OS標準のキーチェーン（Windows Credential ManagerやmacOS Keychainなど）に認証情報を暗号化して保存するため、パスワードを平文で管理する必要がない。
    - 個人アクセストークン（PAT）の自動生成: ユーザーが手動でトークンを発行・コピー＆ペーストしなくても、ログイン操作だけで一時的なトークンを自動で取得・利用する。
    - GitHub Copilotとの親和性: GitHub Enterprise Cloud (EMU) 環境など、高度な認証が求められる環境でGitを操作する際のデファクトスタンダードとなっている。
  - 導入メリット:
    - 認証エラーによる作業の中断を防ぎ、複雑な認証設定（SSO等）を意識することなくコマンドラインからのGit操作を可能にする。
- **Copilot Extensions**: GitHub Copilot Chatの機能を拡張し、外部サービスと双方向にやり取りすることを可能にするエージェントベースの連携機構。

- 仕組みの仕組み（アーキテクチャ）:
  - 呼び出し（インテント解析）: ユーザーがChatで @extension-name と入力すると、GitHubがその意図を解釈します。
  - リクエストの転送: GitHubは、その拡張機能をホストしている外部サーバー（App）に対して、ユーザーの質問や文脈をHTTPSリクエストとして送信します。
  - 外部処理: 拡張機能側（API）で、自社データの検索やツールの操作（例：デプロイ、チケット作成）を実行し、回答を生成します。
  - 回答の表示: 外部サーバーからのレスポンスをGitHub Copilotが受け取り、リッチなテキストやUIとしてユーザーに表示します。
- 主な特徴:
  - スキルセットの追加: 「Sentryでエラーログを調べる」「Azureにリソースを作成する」といった、AIモデル単体では不可能な「アクション（実行）」が可能になります。
  - GitHub Appとしての実装: Extensionsは GitHub App として構築されるため、認証や権限管理が厳格に行われます。
  - カスタム拡張の作成: 企業は自社専用の拡張機能（Internal Extension）を開発し、社内のドキュメント検索や独自のデプロイフローと連携させることができます。
- 導入メリット:
  - コンテキストの切り替え（コンテキストスイッチ）を最小限に抑え、開発に必要なすべての操作をチャットインターフェースに統合できる。

---

# 付録A：参照資料（公式・準公式・ベンダー提供）

## A-1. 公式（GitHub Docs / GitHub Resources / GitHub Blog）

- **Driving GitHub Copilot adoption in your company**（展開・定着プロセス）
  https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption) docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption)
- **Measuring the impact of GitHub Copilot（ESSP）**（測定フレームワーク）
  https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/ (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/) resources.github.com (https://resources.github.com/learn/pathways/copilot/essentials/measuring-the-impact-of-github-copilot/)
- **GitHub Well-Architected – Copilot**（設計原則：生産性/ガバナンス/セキュリティ等）
  https://wellarchitected.github.com/features/copilot/ (https://wellarchitected.github.com/features/copilot/) wellarchit…github.com (https://wellarchitected.github.com/features/copilot/)
- **Managing policies and features for GitHub Copilot in your enterprise**（AI Controls/ポリシー）
  https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies) docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies)
- **Content exclusion for GitHub Copilot**（除外の仕様と制限）
  https://docs.github.com/en/copilot/concepts/context/content-exclusion

(https://docs.github.com/en/copilot/concepts/context/content-exclusion) docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion)

- **GitHub Copilot usage metrics**（メトリクス：ダッシュボード/API/NDJSON、範囲・制限） https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics) docs.github.com (https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics)
- **Viewing the Copilot usage metrics dashboard**（見方・運用） https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption) docs.github.com (https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-enterprise/view-usage-and-adoption)
- **Prompt engineering for GitHub Copilot Chat**（プロンプト指針） https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering) docs.github.com (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering)
- **GitHub Copilot Trust Center（日本語ページ含む）**（セキュリティ/プライバシー/IP/透明性） https://resources.github.com/ja/copilot-trust-center/ (https://resources.github.com/ja/copilot-trust-center/) https://github.blog/news-insights/policy-news-and-insights/how-to-responsibly-adopt-github-copilot-with-the-github-copilot-trust-center/ (https://github.blog/news-insights/policy-news-and-insights/how-to-responsibly-adopt-github-copilot-with-the-github-copilot-trust-center/) https://copilot.github.trust.page/faq (https://copilot.github.trust.page/faq) resources.github.com (https://resources.github.com/ja/copilot-trust-center/) github.blog (https://github.blog/news-insights/policy-news-and-insights/how-to-responsibly-adopt-github-copilot-with-the-github-copilot-trust-center/) copilot.gi…trust.page (https://copilot.github.trust.page/faq)
- **Copilot usage metrics dashboard and API in public preview**（プレビュー告知・範囲） https://github.blog/changelog/2025-10-28-copilot-usage-metrics-dashboard-and-api-in-public-preview/ (https://github.blog/changelog/2025-10-28-copilot-usage-metrics-dashboard-and-api-in-public-preview/) github.blog (https://github.blog/changelog/2025-10-28-copilot-usage-metrics-dashboard-and-api-in-public-preview/)

- **How to write better prompts for GitHub Copilot**（実例中心のプロンプト改善）
  https://github.blog/developer-skills/github/how-to-write-better-prompts-for-github-copilot/ (https://github.blog/developer-skills/github/how-to-write-better-prompts-for-github-copilot/) github.blog (https://github.blog/developer-skills/github/how-to-write-better-prompts-for-github-copilot/)

### A-2. 準公式/コミュニティ（実務者の枠組み）

- **Minimum Required Tasks for Copilot Adoption**（大規模導入チェックリスト）
  https://samqbush.github.io/copilot-adoption/ (https://samqbush.github.io/copilot-adoption/) samqbush.github.io (https://samqbush.github.io/copilot-adoption/)
- **Awesome GitHub Copilot**（コミュニティのプロンプト/設定資産）
  https://github.com/github/awesome-copilot (https://github.com/github/awesome-copilot) github.com (https://github.com/github/awesome-copilot)

### A-3. ベンダー提供 / 解説記事（参考：非公式）

- **Atmosera – GitHub Copilot Adoption Framework**（"フレームワーク"を名乗るベンダー資料）
  https://www.atmosera.com/accelerate/github-copilot-adoption-framework/ (https://www.atmosera.com/accelerate/github-copilot-adoption-framework/) atmosera.com (https://www.atmosera.com/accelerate/github-copilot-adoption-framework/)
- **Microsoft Dev Blog：Adopting Copilot at Scale**（大規模導入のTips）
  https://devblogs.microsoft.com/all-things-azure/adopting-github-copilot-at-scale/ (https://devblogs.microsoft.com/all-things-azure/adopting-github-copilot-at-scale/) devblogs.m...rosoft.com (https://devblogs.microsoft.com/all-things-azure/adopting-github-copilot-at-scale/)
- **Microsoft Dev Blog：Content Exclusion**（除外機能の解説）
  https://devblogs.microsoft.com/cppblog/configure-github-copilot-access-via-content-exclusion/ (https://devblogs.microsoft.com/cppblog/configure-github-copilot-access-via-content-exclusion/) devblogs.m...rosoft.com (https://devblogs.microsoft.com/cppblog/configure-github-copilot-access-via-content-exclusion/)
- **Copilot security controls（TechCommunity）**（導入懸念の説明補助）
  https://techcommunity.microsoft.com/blog/azuredevcommunityblog/demystifying-github-copilot-security-controls-easing-concerns-for-organizational/4468193 (https://techcommunity.microsoft.com/blog/azuredevcommunityblog/demystifying-github-copilot-security-controls-easing-concerns-for-organizational/4468193) techcommun...rosoft.com

(https://techcommunity.microsoft.com/blog/azuredevcommunityblog/demystifying-github-copilot-security-controls-easing-concerns-for-organizational/4468193)

- **Governance Model for Enterprises（解説）** https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/ (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/) c-sharpcorner.com (https://www.c-sharpcorner.com/article/github-copilot-governance-model-for-enterprises/)
- **copilot-metrics-dashboard（アクセラレータ）** https://github.com/microsoft/copilot-metrics-dashboard (https://github.com/microsoft/copilot-metrics-dashboard) github.com (https://github.com/microsoft/copilot-metrics-dashboard)

---

## 付録B：Image一覧

1. **全体像（6視点の1枚絵）**
   - 目的: 全体構造を一目で把握する
   - 画像: `6視点の1枚絵（CAFライク）.png`
2. **導入ロードマップ（フェーズ図）**
   - 目的: 実施順序と期間を明確化する
3. **現状 vs 目標（As-Is / To-Be比較図）**
   - 目的: 変化点と期待効果を示す
4. **組織体制図（役割・責任範囲）**
   - 目的: 担当者と責任分界を明確化する
5. **業務フロー図（Before/After）**
   - 目的: プロセス改善点を可視化する
6. **システム構成図（論理アーキテクチャ）**
   - 目的: ツール・データ連携関係を整理する

7. **ユースケース例（画面キャプチャ付き）**
    - 目的: 利用シナリオを具体化する
8. **KPIダッシュボード例（成果指標）**
    - 目的: 効果を定量的に示す
9. **リスクと対策マップ**
    - 目的: 想定課題と対策方針を共有する
10. **FAQ/よくある誤解の図解**
    - 目的: 読者の疑問を事前に解消する

---

# 次にできること（おすすめ）

このGCAFを「社内標準」にするなら、次の2点を追加すると一気に実戦投入できます。

1. **社内向けテンプレー式**（Word/Confluence想定）

- 利用ポリシーひな型（禁止事項、責任分界、例外手続き）
- Content Exclusion 推奨パターン集
- レビュー観点チェックリスト
- 30/60/90日導入計画書

2. **PowerPoint版（AWS CAF風の構成）**

- 6視点の1枚絵（CAFライク）
- フェーズ別ロードマップ
- KPI/ダッシュボード例

---

## 質問（最小限）

GCAFをあなたの組織に合わせて「完成版」にするため、次だけ教えてください。

1. 対象は **Copilot Business / Copilot Enterprise** のどちら想定ですか？（ポリシー・機能差があるため） docs.github.com (https://docs.github.com/copilot/how-tos/administer-copilot/manage-for-enterprise/manage-enterprise-policies), docs.github.com (https://docs.github.com/en/copilot/concepts/context/content-exclusion)
2. まずの対象は **全社**ですか？それとも **特定部門（例：開発部門のみ）**ですか？ docs.github.com (https://docs.github.com/en/copilot/tutorials/roll-out-at-scale/enable-developers/drive-adoption), samqbush.github.io (https://samqbush.github.io/copilot-adoption/)

回答があれば、上記GCAFを **あなたの導入条件に最適化した“社内配布版（章立て・テンプレ付き）”** に仕上げます.

# Governance

## Copilot利用ポリシーひな型（禁止事項・責任分界・例外手続き）

> **関連ドキュメント**: 導入フレームワーク (#section-01-framework-01-overview) | Content Exclusion推奨パターン集 (#section-02-governance-02-content-exclusion) | レビュー観点チェックリスト (#section-03-operations-01-review-checklist)

- 文書番号：`[POL-COPILOT-001]`
- 版数：`v0.1（初版）`
- 制定日：`[YYYY-MM-DD]`
- 施行日：`[YYYY-MM-DD]`
- 所管：`[情報システム部 / 開発統括部 / セキュリティ統括]`
- 対象：`[Copilot Business / Copilot Enterprise]`

---

### 1. 目的

本ポリシーは、GitHub Copilotを安全かつ効果的に利用するために、禁止事項、責任分界、例外手続きを定める。

### 2. 適用範囲

- 対象者：`[自社従業員 / 委託先 / 派遣 / その他]`
- 対象システム：`[社内開発リポジトリ / 顧客向け案件 / 検証環境 など]`
- 対象ツール：GitHub Copilot（IDE補完、Chat、関連機能）

### 3. 基本原則

1. Copilotは開発支援ツールであり、最終判断は利用者が行う。
2. AI生成コードも手書きコードと同等にレビュー・テストを実施する。
3. 機密情報・個人情報・顧客情報の保護を最優先とする。

4. 利用ログ・設定変更は監査可能な形で記録する。

## 4. 禁止事項

### 4.1 入力禁止（プロンプト・貼り付け・参照指示）

以下の情報をCopilotへ入力してはならない。- 秘密情報（APIキー、パスワード、証明書秘密鍵、アクセストークン） - 個人情報（氏名、住所、電話、メール、マイナンバー等） - 顧客機密情報（契約情報、未公開設計、障害詳細、業務データ） - 法令・契約で外部提供が制限される情報 - 本番データの生値（マスキングされていないデータ）

### 4.2 利用禁止行為

- セキュリティ要件を満たさないコードの無検証採用
- ライセンス不明コードの無確認流用
- レビューを省略した本番反映
- Content Exclusion対象の回避・無効化
- 監査ログの改ざん・削除

### 4.3 条件付き利用（要追加承認）

以下は原則禁止ではないが、事前承認を要する。- 認証・認可・暗号・決済・安全性クリティカル領域の実装支援 - 規制対象プロジェクト（金融・医療・公共等） - 外部委託先環境でのCopilot利用

## 5. 責任分界

### 5.1 利用者（開発者）

- 入力データの適法性・機密性を確認する責任を負う。
- 生成結果の妥当性・安全性・保守性を検証する責任を負う。
- 不明点や高リスク利用時は上長/セキュリティ窓口に相談する。

### 5.2 レビュアー/Tech Lead

- AI生成コードを含むPRの品質・セキュリティ・設計妥当性を審査する。
- 例外利用の妥当性と代替策有無を確認する。

### 5.3 管理者（Org/Enterprise Admin）

- AI Controls、ライセンス割当、Content Exclusion等を適切に設定・運用する。
- 利用状況メトリクスを定期確認し、逸脱を是正する。

### 5.4 セキュリティ/法務

- 禁止事項・例外条件・監査要件を定義/更新する。
- 重大インシデント発生時の調査・是正を主導する。

### 5.5 最終責任

- 成果物（コード・設計・ドキュメント）の最終責任は開発組織にある。
- Copilotベンダーは成果物の業務適合性を保証しない。

## 6. 例外手続き（高リスク領域・特例運用）

### 6.1 申請条件

次のいずれかに該当する場合、例外申請を必須とする。 - 4.3の条件付き利用に該当 - 既定の禁止事項に抵触する可能性がある - 顧客契約・法令上の追加要件がある

### 6.2 申請者が提出する情報

- 申請者情報（部署、氏名、連絡先）
- 対象プロジェクトと対象範囲
- 例外が必要な理由（業務上の必要性）
- 想定リスクと低減策（マスキング、限定公開、追加レビュー等）
- 期間（開始日・終了日）
- 代替策の検討結果（なぜ通常運用では不可か）

### 6.3 承認フロー（例）

1. 申請者が所定フォームを提出
2. Tech Lead一次審査（技術妥当性）
3. セキュリティ審査（情報取扱・統制）
4. 法務確認（契約/規制の観点がある場合）
5. 承認者決裁（部門長またはCISO委任者）

### 6.4 承認時の必須条件

- 期限付き承認（恒久例外は禁止）
- 追加コントロール設定（Content Exclusion強化、ログ監査頻度増）
- 完了後の事後レビュー（効果・逸脱有無・再発防止）

### 6.5 緊急時の暫定例外

- 障害対応等で緊急利用が必要な場合、口頭/チャット承認で暫定実施可。
- ただし `[24時間以内]` に正式申請へ切替え、事後承認を取得すること。

## 7. 監査・記録

- 申請/承認/却下履歴を `[チケットシステム名]` に保管する。
- ポリシー設定変更履歴を `[管理台帳]` に記録する。
- 監査証跡の保管期間： `[1年 / 3年 / 5年]`

## 8. 違反時対応

- 軽微違反：是正指導・再教育
- 重大違反：利用停止、インシデント対応、懲戒手続き（就業規則に準拠）

## 9. 教育・周知

- 初回利用前に必須教育（30〜60分）を受講する。
- 年1回以上の再教育を実施する。
- FAQ・相談窓口を `[Teams/Slackチャンネル名]` で運用する。

## 10. 見直し

- 見直し頻度： `半年ごと` または重大変更時
- 見直しトリガー：法令改正、契約変更、重大事故、機能仕様変更

---

## 付録A：例外申請書テンプレ（貼り付け用）

### A-1. 基本情報

- 申請日：
- 申請者：
- 部署：
- プロジェクト名：
- 対象リポジトリ：

### A-2. 例外内容

- 対象機能（IDE補完 / Chat / その他）：
- 例外が必要な理由：
- 対象データ種別：
- 利用期間：

### A-3. リスクと対策

- 想定リスク：
- 低減策：
- 追加レビュー方法：

### A-4. 承認

- Tech Lead：
- セキュリティ：
- 法務（必要時）：
- 最終承認者：

---

## 付録B：運用チェック（最小）

- [ ] 禁止事項の周知完了
- [ ] Content Exclusion設定済み

- [ ] 例外申請フロー公開済み
- [ ] 監査ログ保管先定義済み
- [ ] 初回教育実施済み

## Content Exclusion 推奨パターン集（GitHub Copilot向け）

> **関連ドキュメント**: 導入フレームワーク (#section-01-framework-01-overview) | ポリシーひな型 (#section-02-governance-01-policy-template)

- 文書番号：`[SEC-COPILOT-CE-001]`
- 版数：`v0.1`
- 最終更新日：`[YYYY-MM-DD]`
- 対象：`Copilot Business / Enterprise`

---

### 1. 目的

本書は、GitHub CopilotのContent Exclusion設定を安全かつ実務的に適用するための推奨パターンを定義する。

### 2. 適用方針（基本）

1. **まず除外すべきものを明確化**：秘密情報、個人情報、顧客機密を優先。
2. **最小限から開始**：過剰除外で生産性を下げない。
3. **リポジトリ単位で差分管理**：全社共通＋案件固有ルールを分離。
4. **定期見直し**：四半期ごと、または事故・監査指摘時に更新。

### 3. 推奨パターン（共通・必須）

以下は原則として全社共通で除外する。

```
(#cb1-1)# 機密情報ファイル
(#cb1-2)**/.env
(#cb1-3)**/.env.*
(#cb1-4)**/*.pem
(#cb1-5)**/*.key
(#cb1-6)**/*.p12
(#cb1-7)**/*.pfx
(#cb1-8)**/id_rsa
(#cb1-9)**/id_ed25519
(#cb1-10)**/known_hosts
(#cb1-11)
(#cb1-12)# クラウド/認証情報
(#cb1-13)**/.aws/**
(#cb1-14)**/.azure/**
(#cb1-15)**/.kube/**
(#cb1-16)**/secrets*.json
(#cb1-17)**/*secret*.json
(#cb1-18)**/*credentials*.json
(#cb1-19)
(#cb1-20)# CI/CD・配布系の機密
(#cb1-21)**/.npmrc
(#cb1-22)**/.pypirc
(#cb1-23)**/nuget.config
(#cb1-24)**/NuGet.Config
(#cb1-25)**/docker-compose.override*.yml
(#cb1-26)
(#cb1-27)# 証跡・ダンプ
(#cb1-28)**/*dump*.sql
(#cb1-29)**/*.bak
(#cb1-30)**/*.backup
(#cb1-31)**/*_backup_*
(#cb1-32)**/*_export_*
(#cb1-33)
(#cb1-34)# 鍵/証明書の格納ディレクトリ
(#cb1-35)**/cert/**
(#cb1-36)**/certs/**
(#cb1-37)**/keys/**
```

## 4. 推奨パターン（条件付き）

以下はプロジェクト特性に応じて追加する。

```
(#cb2-1)# 契約・法務資料
(#cb2-2)**/contract/**
(#cb2-3)**/legal/**
(#cb2-4)**/*契約*.*
(#cb2-5)**/*NDA*.*
(#cb2-6)
(#cb2-7)# 要件・設計の機微情報
(#cb2-8)**/requirements/**
(#cb2-9)**/architecture/**
(#cb2-10)**/*要件定義*.*
(#cb2-11)**/*基本設計*.*
(#cb2-12)
(#cb2-13)# 顧客データ・移行データ
(#cb2-14)**/customer-data/**
(#cb2-15)**/migration-data/**
(#cb2-16)**/*顧客*.*
(#cb2-17)**/*個人情報*.*
(#cb2-18)
(#cb2-19)# 監査/脆弱性報告
(#cb2-20)**/audit/**
(#cb2-21)**/pentest/**
(#cb2-22)**/*脆弱性*.*
(#cb2-23)**/*security-report*.*
```

## 5. C# / .NET プロジェクト向け追加推奨

```
(#cb3-1)# アプリ設定（環境依存）
(#cb3-2)**/appsettings.Production.json
(#cb3-3)**/appsettings.Staging.json
(#cb3-4)**/appsettings.Local.json
(#cb3-5)
(#cb3-6)# ユーザーシークレットやローカル設定
(#cb3-7)**/secrets.json
(#cb3-8)**/*local.settings.json
(#cb3-9)
(#cb3-10)# 配布前構成・署名関連
(#cb3-11)**/*.snk
(#cb3-12)**/*SignKey*.*
```

## 6. データベース運用向け追加推奨

```
(#cb4-1)# DB接続情報・運用情報
(#cb4-2)**/*connection*.*
(#cb4-3)**/*conn*.*
(#cb4-4)**/*dbconfig*.*
(#cb4-5)**/*postgresql.conf
(#cb4-6)**/*pg_hba.conf
(#cb4-7)
(#cb4-8)# 運用ログ・実データ抽出
(#cb4-9)**/logs/**
(#cb4-10)**/*.log
(#cb4-11)**/*pgdiag*.*
(#cb4-12)**/*process_postgres*.csv
```

## 7. 除外ルール設計のアンチパターン

- `**/*.json` のような広すぎる除外（有用なコード文脈まで失う）
- 一時対応の除外を恒久化して放置
- 例外運用（緊急解除）時の証跡未記録
- ルール変更時に開発者へ周知しない

## 8. 運用手順（最小）

1. セキュリティ担当が初期パターン案を作成。
2. Tech Leadが開発影響をレビュー。
3. 管理者がOrg/Enterpriseへ反映。
4. 反映後、1週間は問い合わせ窓口を開設。
5. 月次で「追加/削除候補」を棚卸し。

## 9. 例外手続き（Content Exclusion個別）

- 申請対象：除外解除、限定解除、期間限定解除
- 必須情報：対象パス、理由、期間、代替策、承認者
- 承認要件：Tech Lead + セキュリティ担当の両承認
- 期限：最長 `30日` 、延長時は再申請

## 10. チェックリスト

- ☐ 秘密情報系パターンを適用済み
- ☐ 顧客機密系パターンを適用済み
- ☐ C#/.NET固有設定を確認済み
- ☐ DB運用ファイルの扱いを確認済み
- ☐ 例外申請フローを公開済み
- ☐ 見直し日（四半期）を設定済み

---

## 付録A：導入初期セット（コピペ用）

```
(#cb5-1)**/.env
(#cb5-2)**/.env.*
(#cb5-3)**/*.pem
(#cb5-4)**/*.key
(#cb5-5)**/*.p12
(#cb5-6)**/*.pfx
(#cb5-7)**/.aws/**
(#cb5-8)**/.azure/**
(#cb5-9)**/.kube/**
(#cb5-10)**/secrets*.json
(#cb5-11)**/*credentials*.json
(#cb5-12)**/appsettings.Production.json
(#cb5-13)**/appsettings.Staging.json
(#cb5-14)**/secrets.json
(#cb5-15)**/*postgresql.conf
(#cb5-16)**/*pg_hba.conf
(#cb5-17)**/logs/**
(#cb5-18)**/*.log
```

## 付録B：変更履歴

- `[YYYY-MM-DD]` v0.1 初版作成

# Operations

## レビュー観点チェックリスト（GitHub Copilot利用時）

> **関連ドキュメント**: 導入フレームワーク (#section-01-framework-01-overview) | ポリシーひな型 (#section-02-governance-01-policy-template) | Content Exclusion推奨パターン集 (#section-02-governance-02-content-exclusion)

- 文書番号：`[REV-COPILOT-001]`
- 版数：`v0.1`
- 最終更新日：`2026-03-21`
- 対象：`C# / .NET プロジェクト（必要に応じて他言語へ横展開）`

---

### 1. 利用前提の確認

- [ ] 本変更はCopilot利用ポリシーの適用範囲内である
- [ ] 禁止入力（秘密情報・個人情報・顧客機密）をプロンプトに含めていない
- [ ] Content Exclusion対象ファイルを回避していない
- [ ] 例外利用が必要な場合、事前承認または暫定承認の記録がある

### 2. 変更の妥当性（要求適合）

- [ ] 要件/仕様に対して過不足なく実装されている
- [ ] 仕様外の機能追加・挙動変更がない
- [ ] 既存設計（命名、責務分離、レイヤ構成）に整合している
- [ ] 既存API/契約（I/F、DTO、レスポンス形式）を破壊していない

### 3. 可読性・保守性

- [ ] コードが簡潔で、過剰な複雑化がない

- [ ] 変数名・メソッド名が意図を表している
- [ ] 重複コードが抑制され、再利用可能な形になっている
- [ ] 例外的ロジックに必要な説明（コメント/命名）がある

### 4. セキュリティ

- [ ] 入力値検証（null/範囲/形式/長さ）が適切に実装されている
- [ ] 認可/認証要件（ロール、権限制御）を満たしている
- [ ] SQLインジェクション、コマンドインジェクション、XSS等の懸念がない
- [ ] 秘密情報がソース/ログ/例外メッセージに露出していない
- [ ] 暗号・署名・鍵管理の実装が社内標準に準拠している

### 5. データ整合性・業務整合

- [ ] トランザクション境界が適切で整合性を損なわない
- [ ] 並行実行時の競合（ロック、再試行、冪等性）に配慮されている
- [ ] 日時・タイムゾーン・桁あふれ等の境界条件を考慮している
- [ ] 既存データへの影響（移行、互換性、削除条件）を評価している

### 6. パフォーマンス

- [ ] 不要なループ/再計算/同期I/Oがない
- [ ] DBアクセス回数とクエリ効率（N+1、索引利用）を確認した
- [ ] 大量データ時の挙動（メモリ使用量、タイムアウト）を考慮した
- [ ] 必要に応じて計測結果（ベースライン比較）を添付している

### 7. 例外処理・ログ

- [ ] 例外を握り潰していない（適切に再送出または変換）
- [ ] 利用者向けメッセージと運用者向けログを分離している
- [ ] ログレベル（Info/Warn/Error）が適切で過不足がない
- [ ] 個人情報・機密情報をログに出力していない

### 8. テスト

- ☐ 単体テストが追加/更新され、変更点をカバーしている
- ☐ 失敗系・境界値・例外系テストが含まれている
- ☐ 既存テストが全て通過している
- ☐ 回帰リスクが高い箇所に追加検証（結合/手動確認）を実施した

### 9. 依存関係・ライセンス

- ☐ 新規依存パッケージの必要性が説明されている
- ☐ ライセンス上の問題がない
- ☐ 脆弱性既知パッケージを導入していない
- ☐ バージョン固定方針・更新方針に準拠している

### 10. PR運用

- ☐ PR説明に「目的・変更内容・影響範囲・検証結果」を記載した
- ☐ AI利用の有無とレビュー上の注意点を明示した
- ☐ 影響範囲（関連モジュール/運用手順）を関係者へ共有した
- ☐ マージ条件（承認数、CI通過、セキュリティチェック）を満たした

---

## 付録A：AI生成コード向け追加確認（任意）

- ☐ 生成コードをそのまま採用せず、意図を理解したうえで手直しした
- ☐ 外部由来コードの類似性/著作権リスクを確認した
- ☐ ドメイン固有ルール（社内規約、監査要件）へ適合させた
- ☐ プロンプトで不足した前提条件をPRで補足した

## 付録B：レビュー記録テンプレ

- レビュー日：
- 対象PR：
- レビュアー：
- 判定（承認 / 条件付き承認 / 差戻し）：

- 主な指摘事項：
- 再確認ポイント：

---

### 運用メモ

- 本チェックリストは「網羅性」と「実用性」の両立を狙った標準版。
- 重要案件では、セキュリティ/法務/運用の追加レビュー項目を追補すること。

## チャット参加者・スラッシュコマンド・チャット変数の詳細リスト

> **関連ドキュメント**: 導入フレームワーク (#section-01-framework-01-overview) | レビュー観点チェックリスト (#section-03-operations-01-review-checklist)

**概念的な位置づけ：**

| 階層 | 概念 | 例 | 役割 |
|---|---|---|---|
| **Who（誰が）** | チャット参加者 | `@github`, `@vscode`, `@extension-name` | ChatコンテキストでAIが「特定のスキル」「エージェント」を起動 |
| **What（何をするか）** | スラッシュコマンド | `/explain`, `/test`, `/fix`, `/doc`, `/help` | AIが「特定の処理」を実行（説明・テスト生成・修正など） |
| **Where/How（どこを/どう対象にするか）** | チャット変数 | `@workspace`, `#file`, `#selection`, `@symbol` | AIが「参照範囲」「文脈の絞り込み」を実行 |

---

### 1. チャット参加者（Chat Participants）

Copilot Chat内で、**特定のエージェントやスキルセットを起動**する記法。`@` 記号で指定。

**主なチャット参加者**

- **@github**：GitHub固有のコンテキスト（リポジトリ、PR、Issue、REST API等）を備えたエージェント。GitHubプラットフォーム理解が必要なタスク向け。
- **@vscode**：Visual Studio Codeの機能・設定・拡張機能に関する質問専用。IDE操作やワークスペース設定の相談に最適。
- **@azure**（Azure Copilot統合の場合）：Azureの各種サービス（VM、App Service、Cognitive Services等）の構築・管理に関する相談。
- **@jira**（Copilot Extensions経由の場合）：Jiraチケット・プロジェクト管理システムへのアクセス。タスク履歴の参照やチケット作成が可能。
- **@slack**（拡張機能が有効な場合）：Slackワークスペースの検索・メッセージ検索。チーム内情報の統合利用。
- **@terminal**：ターミナル・コマンドラインに関する質問専用。シェルコマンド、スクリプト、ターミナル操作のトラブルシューティング向け。
- **@extension-name**（カスタム拡張）：企業開発のCopilot Extensionを指定。社内ドキュメント検索、独自APIへのアクセス、デプロイシステム連携など、組織固有のスキルセットを追加可能。

**活用パターン**

```
@github でこのリポジトリの依存関係を分析して
 ->  GitHubコンテキストを活用し、dependencies解析

@vscode で現在のワークスペース設定を説明して
 ->  IDE設定のクイックリファレンス

@jira チケット PROJ-456 の内容から実装方針を提案して
 ->  外部システム連携でタスク内容を自動取得
```

---

## 2. スラッシュコマンド（Slash Commands）

Copilot Chat内で、**AIに「何をするか」という定型的な処理を指示**する短縮コマンド。`/` 記号で開始。

### 標準的なスラッシュコマンド

- **/explain**：選択コード、メソッド、ファイル全体の説明。何をしているコードかを自然言語で解説。
- **/test**：選択コードの単体テスト（ユニットテスト）コード生成。テストフレームワーク（Jest, pytest等）に自動対応。
- **/fix**：エラーメッセージやバグを指摘された箇所を自動修正。コード品質の改善提案も含む。
- **/doc**：選択コードのドキュメント自動生成。JSDoc、XML Doc、Docstringなど言語別フォーマットで出力。
- **/help**：Copilot Chatの使い方、利用可能なコマンド・参加者一覧を表示。
- **/clear**：現在のチャット履歴をリセット。新しい会話を開始する際に利用。
- **/new**：独立した新規チャットセッションを開始（IDE等で対応している場合）。
- **/newNotebook**：新しいJupyterノートブックをセットアップ。データ分析やデータ探索用のテンプレート生成。
- **/delete**：現在のチャット会話を削除。不要なチャット履歴の消去に利用。
- **/rename**：現在のチャット会話の名前を変更。チャット管理と整理に便利。
- **/tests**：選択コードの単体テスト（ユニットテスト）コード生成。`/test` と同じ機能で、フレームワーク自動対応。

### 活用パターン

```
/explain 選択コード
 ->  複雑な関数の意図を素早く理解

/test
 ->  テストがないメソッドに対してテストコードを生成

/fix コピペしたコードのエラーを修正
 ->  Stack Overflowからのコード＆バグを一度に処理

/doc APIメソッド
 ->  OpenAPI / Swagger対応のドキュメント生成
```

### 3. チャット変数（Chat Variables）

Copilot Chat内で、「**どのファイル/スコープ/リソースを対象にするか**」**という参照範囲を指定**する記法。`@` または `#` 記号で指定。

**ワークスペース・ファイル関連**

- **@workspace**：プロジェクト全体のコンテキストをAIに含める。ファイル構造、依存関係、設定ファイル等を自動分析。
  - 用途：アーキテクチャ評価、大規模リファクタリング計画、プロジェクト全体の説明
- **#file**：ワークスペース内の特定のファイルを参照します。ファイル選択UIが表示されるため、視覚的に選べます。
  - 用途：バグ修正、特定モジュールの説明、そのファイル内での最適化
- **#selection**：エディタで現在選択しているコードの箇所を参照します。
  - 用途：関数単位の修正、メソッドの説明、小規模なコードレビュー
- **#editor**：アクティブなエディタで表示されている範囲全体を参照します。
  - 用途：ウィンドウに表示されている全ファイル内容を対象にした分析や説明

**シンボル・参照関連**

- **@symbol** または **@関数名**：リポジトリ内の特定シンボル（関数・クラス・メソッド等）を指定（IDE機能依存）。
  - 用途：関数の呼び出し元を把握する、特定クラスの設計を理解する
- **@class ClassName**：特定クラスおよびそのメンバーをコンテキストに指定（IDE機能依存）。

**コード検索・分析関連**

- **#codebase**：プロジェクト全体のコードベースを対象に意味検索（セマンティックサーチ）を行います。
  - 用途：特定の機能実装の探索、設計パターンの検出、テクニカルデブトの把握

**拡張・外部サービス関連**

- **@extension-name**：Copilot Extensionsで接続した外部サービスを参照。データベース、内部Wiki、デプロイシステム等を統合。
  - 用途：社内ドキュメント検索、環境固有の設定確認、ツール連携

**ターミナル関連**

- **terminal_last_command**：アクティブなターミナルで最後に実行したコマンドとその出力結果を参照します。
  - 用途：実行結果の分析、エラーメッセージに基づく修正、トラブルシューティング
- **terminal_selection**：ターミナルで現在選択されている箇所を参照します。
  - 用途：ターミナル出力の特定部分をコンテキストに含める、ログ分析

**開発コンテキスト関連**

- **get_changed_files**：Gitにおける最新のソース管理の変更点（未ステージングの変更など）を参照します。
  - 用途：変更内容の確認、コミットメッセージの自動生成、変更影響度の分析
- **get_errors**：VS Codeの「問題」パネルに表示されているエラーや警告を参照します。デバッグ時に便利です。
  - 用途：エラーの一括修正提案、コードの問題把握、品質向上
- **test_failure**：前回の単体テスト実行時の失敗に関する情報を参照します。
  - 用途：テスト失敗の原因分析、修正案の提案、テストコードの改善

**外部情報関連**

- **fetch_webpage**：指定したURLのWebページの内容を取得して参照します。
  - 用途：オンラインドキュメント参照、外部APIドキュメント確認、技術情報の統合
- **#websearch**：（拡張機能が必要）Web検索エンジン（BingやTavily）の結果を参照します。

    - 用途：外部情報検索、技術トレンド調査、問題解決のためのネット検索
- **#web**：@github との組み合わせで使用。Webの最新情報を検索してコンテキストに含めます。
    - 用途：最新の技術情報、LTSバージョン確認、トレンド情報の取得
- **#git**：Gitリポジトリの履歴、ブランチ情報、変更履歴を参照します。
    - 用途：コミット履歴分析、ブランチ管理、変更差分の確認
- **#terminalLastCommand**：ターミナルで最後に実行したコマンドとその出力をより詳細に参照します。
    - 用途：コマンド実行結果の詳細分析、複雑なエラーの診断
- **#runSubagent**：Copilot Chat内で独立したサブエージェントを実行。複雑なマルチステップタスクを委譲。
    - 用途：複数ファイルの統合分析、大規模な研究・分析タスク、コンテキスト分離が必要な処理

### 活用パターン

```
# プロジェクト全体理解
@workspace のテスト構造を図示して
 ->  プロジェクト全体のテスト配置を可視化

# 特定ファイルに絞った処理
#auth.js のセキュリティ脆弱性をチェック
 ->  そのファイルのみを対象にセキュリティレビュー

# 複数ファイル指定による分析
#files:auth.js #files:config.js でこれらファイル間の依存関係を説明
 ->  複数指定したファイル間の統合分析

# エディタ表示範囲を対象にした処理
#editor の型定義を改善して
 ->  現在表示されているエディタ内容全体を対象に改善

# 最小限のコード質問
選択コード片で /explain #selection
 ->  数行のコードだけを対象に説明

# コードベース全体で検索
#codebase でデータベース接続の実装パターンを探して
 ->  プロジェクト全体の意味検索で実装パターンを発見

# ターミナル出力に基づく修正
/fix terminal_last_command
 ->  直前のコマンド実行でエラーが出た場合に修正提案

# Git変更点に基づく処理
get_changed_files のファイルに対してテストを生成して
 ->  未ステージング変更ファイル全体のテストを自動生成

# エラー一括修正
/fix get_errors
 ->  VS Code「問題」パネルのエラーを全て修正提案

# テスト失敗の分析
test_failure の原因を分析して修正案を提案して
 ->  直前のテスト失敗の根本原因と修正方法を提案

# 外部リソース活用
@company-docs でこのプロジェクト用のスタイルガイドを探して
 ->  企業ドキュメント拡張から関連ガイドを検索
```

```
# Webの最新情報を参照
このエラーメッセージを #websearch で検索して解決方法を提案
 ->  オンライン検索で最新の解決方法を提案

# Webページ内容を参照
fetch_webpage で https://docs.example.com を確認して実装ガイドを説明
 ->  指定URLの内容をコンテキストに含めた説明
```

---

## 4. 3つの本質的な違い

| 側面 | チャット参加者 | スラッシュコマンド | チャット変数 |
|---|---|---|---|
| **記号** | `@` | `/` | `@` または `#` |
| **指定方法** | Chat編集欄で入力 | Chat編集欄で/から開始 | Chat編集欄またはエディタから参照 |
| **役割** | 誰が（どのエージェント） | 何をするか（処理の種類） | どこを（参照範囲） |
| **依存関係** | 拡張機能の有無 | IDE/Chat機能の実装 | IDE/Chat機能の実装 |
| **複数指定** | 単数が標準 | 1コマンド/1回 | 複数指定可（複合指定） |
| **前提知識** | Copilot Extensionsの理解 | スラッシュコマンド一覧の習熟 | ワークスペース構造の理解 |

**実践的な組合わせ例：**

```
# 例1：ファイル内の特定コードを修正
/fix #file
 ->  スラッシュコマンド（/fix）で「修正」をリクエストし、
  チャット変数（#file）で「対象をこのファイル」に限定

# 例2：プロジェクト全体で設計相談
@github を見て、このアーキテクチャを説明して
 ->  チャット参加者（@github）で「エージェント起動」し、
  チャット変数（@workspace）で「プロジェクト全体を文脈に」含める

# 例3：外部サービス連携でテスト実行
@jira チケット ABC-123 のテストを実行して
 ->  チャット参加者（@jira）で「拡張機能」を呼び出し、
  スラッシュコマンド（シンプルな命令）で「テスト」を実行

# 例4：/test @workspace でテストをカバレッジ最適化して
 ->  スラッシュコマンド（/test）
  × チャット参加者（@workspace自動適用）
  × チャット変数（プロジェクト全体）

# 例5：@jira チケットABC-123の#js_module を修正して
 ->  チャット参加者（@jira）
  × チャット変数（#ファイル）
  × 暗黙的コマンド（修正）

# 例6：エラー修正と検索の組み合わせ
/fix get_errors で修正が必要な箇所を修正し、残った問題は #websearch で調査
 ->  ローカルエラー修正と外部情報検索を組み合わせた問題解決

# 例7：複数ファイル間の統合分析
#files:service.ts #files:controller.ts の関連性を分析して、リファクタリング提案をして
 ->  複数ファイルを指定して統合分析と改善案を取得

# 例8：変更内容に基づくテスト生成
/test get_changed_files でGit未ステージング変更に対するテストを生成
 ->  実際の変更ファイルを自動検出してテストを生成

# 例9：ターミナル出力の分析と修正
terminal_last_command のエラーを分析して、/fix で修正案をもらう
 ->  コマンド実行結果とコードを組み合わせたトラブルシューティング

# 例10：Webドキュメント連携での実装
fetch_webpage https://docs.api.com/v2 でAPIドキュメント確認後、#codebase の実装を確認し
```

```
て更新提案をもらう
 ->  オンラインドキュメントとコードベースを統合した実装改善
```

---

### 5. チーム導入時のチェックリスト

- ❑ チャット参加者は「エージェント選択」である点を理解している
- ❑ スラッシュコマンドは「IDE標準機能」であり、どのCopilot環境でも動作することを確認
- ❑ 基本的なチャット変数（@workspace、#file、#selection）の使い方を習熟している
- ❑ ファイル関連の拡張チャット変数（#files、#editor、#codebase）の役割を理解している
- ❑ 開発コンテキスト変数（get_changed_files、get_errors、test_failure）の活用方法を確認
- ❑ ターミナル関連変数（terminal_last_command、terminal_selection）をトラブルシューティングで活用できる
- ❑ 外部情報変数（fetch_webpage、#websearch、#web）の必要性と制限を理解している
- ❑ リポジトリコンテキスト変数（#git、#terminalLastCommand）の用途を把握している
- ❑ @terminal チャット参加者がターミナルコマンド対応であることを周知している
- ❑ スラッシュコマンド拡張（/newNotebook、/delete、/rename、/tests）の用途を理解している
- ❑ #runSubagent でマルチステップタスク委譲ができることを確認している
- ❑ 企業カスタムの拡張機能（@extension）が何か、どう使うかを周知している
- ❑ 複数変数と複数コマンドの組み合わせについて具体例で習熟している
- ❑ 社内ドキュメント（ガイド）にこれらのチャット変数を使い分けを明記している

---

## 参考資料


- GitHub Copilot Chat 公式ドキュメント (https://docs.github.com/en/copilot/how-tos/chat-with-copilot/get-started-with-chat)
- Prompt Engineering for GitHub Copilot (https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering)
- GitHub Copilot Extensions (https://docs.github.com/en/copilot/tools/github-copilot-extensions)